Keyword Research Module — Developer Specification

Product: AI Digital Marketing Automation Tool **Module:** Keyword Research **Document type:** Feature & Functionality Spec for Developers **Version:** 1.0

---

1. Purpose

This module allows users to enter a seed keyword, domain, or topic and receive AI-enriched keyword research data: search volume, competition/difficulty, CPC, search intent, related keywords, and clustering — so they can plan SEO/PPC/content campaigns inside the automation tool.

This document is written for the developer(s) who will build this module. It covers features, functionality, data flow, APIs, architecture, and technical requirements.

---

2. Core Features

1. **Seed Keyword Input**

    - User enters one or more seed keywords, a URL, or a competitor domain.
    - System supports bulk upload (CSV/TXT) of seed keywords.

2. **Keyword Expansion (Suggestions)**

    - Generate related keywords, long-tail variations, questions (People Also Ask style), and autocomplete-based suggestions.

3. **Keyword Metrics**

    - Search Volume (monthly, trend over 12 months)
    - Keyword Difficulty (KD) / SEO competition score
    - CPC (Cost Per Click) and PPC competition
    - Search Intent classification (Informational / Navigational / Transactional / Commercial)

4. **AI-Based Clustering**

    - Group keywords into topic clusters/themes using semantic similarity (embeddings), so users can build content pillars automatically.

5. **Competitor Keyword Gap**

    - Compare user's domain vs competitor domain(s) to find keywords competitors rank for but the user doesn't.

6. **SERP Snapshot**

    - Show top 10 ranking pages for a keyword (title, URL, domain authority if available).

7. **Filters & Sorting**

    - Filter by volume range, KD range, intent, word count, CPC, and keyword inclusion/exclusion.

8. **Export & Integration**

    - Export to CSV/Excel/PDF.
    - Push selected keywords directly into other modules (Content Generator, Campaign Planner, Rank Tracker).

9. **Saved Projects / History**

    - Save keyword lists under a project so users can revisit research later.

10. **AI Recommendations**

    - AI suggests "best keywords to target now" based on volume-to-difficulty ratio and user's current site authority.

---

3. Functional Requirements (What the system must do)

ID	Requirement
FR-1	Accept single/bulk keyword input and validate (max length, duplicate removal, language detection)
FR-2	Fetch keyword metrics from a third-party data provider via API
FR-3	Cache metrics data to avoid repeated paid API calls for the same keyword within a TTL window
FR-4	Run keyword expansion using both provider "related keywords" endpoints and Claude/LLM-based semantic expansion
FR-5	Classify search intent using an LLM prompt or a trained classifier
FR-6	Cluster keywords using embeddings + cosine similarity or k-means/HDBSCAN
FR-7	Store all keyword research results per user/project in the database
FR-8	Provide REST/GraphQL API endpoints for frontend and other internal modules to consume
FR-9	Support async/background processing for bulk keyword jobs (queue-based)
FR-10	Rate-limit and quota-manage third-party API usage per user plan (Free/Pro/Agency)
FR-11	Export data in CSV/XLSX/PDF
FR-12	Log all requests for billing/usage analytics

---

4. Suggested Architecture

[Frontend UI]

|

v

[API Gateway / Backend (Node.js / Django / FastAPI)]

|

|---> [Keyword Research Service]

| |---> [3rd Party Data Provider API] (metrics, SERP)

| |---> [LLM Service] (Claude API) - expansion, intent, clustering summaries

| |---> [Cache Layer] (Redis)

| |---> [Database] (Postgres) - projects, saved keywords, users

| |---> [Job Queue] (BullMQ / Celery / RabbitMQ) - for bulk jobs

|

v

[Export Service] --> CSV/XLSX/PDF generation

---

5. Recommended Tech Stack

- **Backend:** Node.js (Express/NestJS) or Python (FastAPI/Django)
- **Database:** PostgreSQL (structured data) + Redis (caching, rate limits)
- **Queue:** BullMQ (Node) or Celery (Python) for async bulk keyword jobs
- **AI/LLM:** Claude API (via Anthropic `/v1/messages`) for:
  - Keyword expansion (semantic, long-tail generation)
  - Search intent classification
  - Cluster labeling / topic naming
  - Content brief generation from keyword clusters

- **Embeddings for clustering:** Any embedding model (OpenAI, Cohere, or self-hosted sentence-transformers) + cosine similarity, or HDBSCAN/k-means
- **Third-party keyword data providers (pick one or combine):**
  - Google Keyword Planner API (via Google Ads API) — free, requires Google Ads account
  - SEMrush API — paid, rich data
  - Ahrefs API — paid, rich data
  - Moz API — paid
  - DataForSEO API — cost-effective, good for search volume/SERP/keyword data
- **Frontend:** React/Next.js (assumed, matches rest of automation tool)
- **Export:** `exceljs`/`pandas` for XLSX, `pdfkit`/`reportlab` for PDF

---

6. Data Model (Suggested Schema)

Table: `keyword_projects` | Field | Type | Notes | |---|---|---| | id | UUID | Primary key | | user_id | UUID | FK to users | | name | string | Project name | | seed_input | text | Original seed keyword(s)/domain | | created_at | timestamp | |

Table: `keywords` | Field | Type | Notes | |---|---|---| | id | UUID | Primary key | | project_id | UUID | FK to keyword_projects | | keyword | string | | | search_volume | integer | Monthly avg | | volume_trend | JSON | 12-month array | | difficulty_score | float | 0–100 | | cpc | float | | | competition | float | 0–1 (PPC competition) | | intent | enum | informational/navigational/transactional/commercial | | cluster_id | UUID | nullable, FK to keyword_clusters | | created_at | timestamp | |

Table: `keyword_clusters` | Field | Type | Notes | |---|---|---| | id | UUID | Primary key | | project_id | UUID | | | cluster_name | string | AI-generated topic label | | created_at | timestamp | |

Table: `api_usage_logs` | Field | Type | Notes | |---|---|---| | id | UUID | | | user_id | UUID | | | endpoint | string | | | provider_calls | integer | For quota tracking | | llm_tokens_used | integer | | | timestamp | timestamp | |

---

7. API Endpoints (Suggested)

POST /api/keyword-research/projects -> create new project

GET /api/keyword-research/projects/:id -> get project + keywords

POST /api/keyword-research/expand -> body: {seed, language, country}

POST /api/keyword-research/bulk -> upload CSV, returns job_id

GET /api/keyword-research/jobs/:job_id -> check bulk job status

POST /api/keyword-research/cluster -> body: {project_id}

POST /api/keyword-research/competitor-gap -> body: {domain, competitor_domain}

GET /api/keyword-research/export/:project_id -> query: format=csv|xlsx|pdf

---

8. LLM Prompt Notes (for Claude API integration)

- **Intent classification:** Send batch of keywords, ask Claude to return strict JSON: `{keyword, intent}` for each. Keep prompt deterministic (low temperature) and enforce JSON-only output.
- **Cluster labeling:** After grouping keywords via embeddings, send each cluster's keyword list to Claude and ask for a short topic name + 1-line description.
- **Long-tail expansion:** Given a seed keyword + industry context, ask Claude to generate N long-tail variants and questions, deduplicated against existing keyword list.
- Always validate LLM JSON output before saving to DB (wrap in try/catch, retry once on malformed JSON).

---

9. Non-Functional Requirements

- **Performance:** Bulk jobs (500+ keywords) must run asynchronously; UI should show progress (%) via polling or websockets.
- **Rate limiting:** Enforce per-plan quotas (e.g., Free = 50 keywords/day, Pro = 2000/day) at the API gateway level.
- **Caching:** Cache third-party metric responses for at least 7–30 days per keyword+country+language combination to reduce provider API cost.
- **Security:** All third-party API keys stored in a secrets manager (not in code/env files committed to repo). Validate/sanitize all user CSV uploads.
- **Scalability:** Queue-based processing so keyword volume spikes don't block the main API.
- **Localization:** Support keyword research by country and language (important for volume/CPC accuracy).

- **Error handling:** Graceful fallback if a provider API fails — show partial data with a "some metrics unavailable" notice rather than failing the whole request.

---

10. What the Developer Needs Before Starting

- ☐ Decision on which third-party keyword data provider(s) to integrate (Google Ads API / DataForSEO / SEMrush / Ahrefs) — affects cost model and available fields
- ☐ Anthropic Claude API key for LLM features (expansion, intent, clustering labels)
- ☐ Embedding model choice for clustering
- ☐ Confirmed subscription tiers/quota limits (Free/Pro/Agency) for rate limiting logic
- ☐ Supported languages/countries for v1 launch
- ☐ Whether competitor-gap analysis and SERP snapshot are in v1 scope or a later phase
- ☐ Design/UX for bulk upload and progress indication
- ☐ Export format priorities (CSV only for v1, or CSV+XLSX+PDF)

---

11. Suggested Build Phases

Phase 1 (MVP): Seed input → provider API metrics → save to DB → basic list UI → CSV export **Phase 2:** Bulk upload + async jobs + intent classification (LLM) + filters/sorting **Phase 3:** AI clustering + cluster labeling + content-brief generation **Phase 4:** Competitor gap analysis + SERP snapshot + push-to-other-modules integration